M.Tech Project Proposal: Predictive-Semantic Monitoring for Decentralized Edge Environments

Literature Review

Tundo et al. (2025) - Decentralized Edge Workload Forecasting With Gossip Learning (IEEE TNSM) - Uses Gossip Learning to train LSTM models across a network to forecast future workloads.
I. Hegedüs, G. Danner, and M. Jelasity, “Gossip learning as a decentralized alternative to federated learning,” in Proc. DAIS, 2019, pp. 74–90 - Gossip Learning as a Decentralized Alternative to Federated Learning
N. Onoszko, G. Karlsson, O. Mogren, and E. L. Zec, “Decentralized federated learning of deep neural networks on non-IID data,” 2021, arXiv:2107.08517 - Performance-Based Neighbor Selection (PENS)

Dataset:

- OpenCellID public dataset: This dataset was utilized to sample real antenna (telecommunication tower) positions to generate the geographical layout of the network nodes.
- Porto taxi dataset: This dataset contains GPS trajectories of taxis operating in Porto, sampled every 15 seconds from 2013 to 2014. Because human movement is a good proxy for network traffic, the authors used these taxi mobility traces to generate realistic traffic profiles and map the function execution requests to specific towers

Base Paper Summary :-

- The paper addresses the volatility of edge workloads by moving away from centralized/Federated Learning (FL) toward Gossip Learning (GL). In GL, nodes (antennas/edge towers) act as equal peers, exchanging LSTM model weights directly with neighbors rather than a central server. This eliminates the "Single Point of Failure" and handles the dynamic, failure-prone nature of edge nodes.
- Performance: GL matches the accuracy of centralized models while providing better generalization than single-node training.
- Robustness: The protocol is highly resilient to node failures; recovered nodes catch up to the "global brain" quickly through asynchronous weight merges.

My Proposed Approach (The Novelty)- Predictive-Semantic Monitoring (PSM)

While the base paper solves the forecasting problem, it still introduces significant bandwidth overhead by constantly exchanging model updates. My approach, Predictive-Semantic Monitoring, uses the forecast to silence the protocol and enable proactive decision-making.

LSTM trains locally; predictions are never used to decide whether to communicate

Innovation 1: Adaptive Semantic Surprise Filter

Instead of periodic gossip, we introduce a trigger based on Semantic Value.

- The Logic: A node only gossips if the "Surprise" (Prediction Error) exceeds a threshold τ.
- Impact: Estimated 40–70% reduction in network packets by suppressing predictable data.

Innovation 2: Bottleneck Signature Evaluator (The "Proactive" Logic)

We move beyond simple forecasting to Decision Making. I define a "Bottleneck Signature" as a future state where:
Current Utilization + Forecasted Workload Increase > 0.85 × System Capacity.

- The Action: When this signature is detected, the node triggers a "Forced Gossip" alert to neighbors.
- Offloading: Task migration starts immediately, while the node still has the CPU headroom to handle the networking overhead.

Mathematical Framework

---

1. Semantic Surprise Metric

Let:

- x(i,t) = actual workload
- x_hat(i,t) = predicted workload

Semantic prediction error:

```
None
ε(i,t) = | x(i,t) − x_hat(i,t) |
```

Where: ε(i,t) = semantic information value

Gossip Decision Function

```
None
G(i,t) = 1 if ε(i,t) > τ
G(i,t) = 0 if ε(i,t) ≤ τ
```

Where:

- G(i,t) = 1 → transmit
- G(i,t) = 0 → remain silent

Adaptive Threshold

```
None
τ = k × σ( ε(t−N ... t) )
```

Where:

- k = sensitivity coefficient
- σ = standard deviation
- N = recent observation window

---

2. Predictive State Proxy (PSP)

If:

```
None
G(i,t) = 0
```

Then node j estimates node i locally:

```
None
x_tilde(j←i,t) = fθ( x_tilde(j←i,t−1) )
```

Where:

- x_tilde = estimated state
- fθ = local LSTM model

Error Bound Guarantee

```
| x(i,t) − x_tilde(i,t) | ≤ τ
```

Meaning: Prediction drift remains bounded.

---

3. Proactive Bottleneck Signature

Future overload detection:

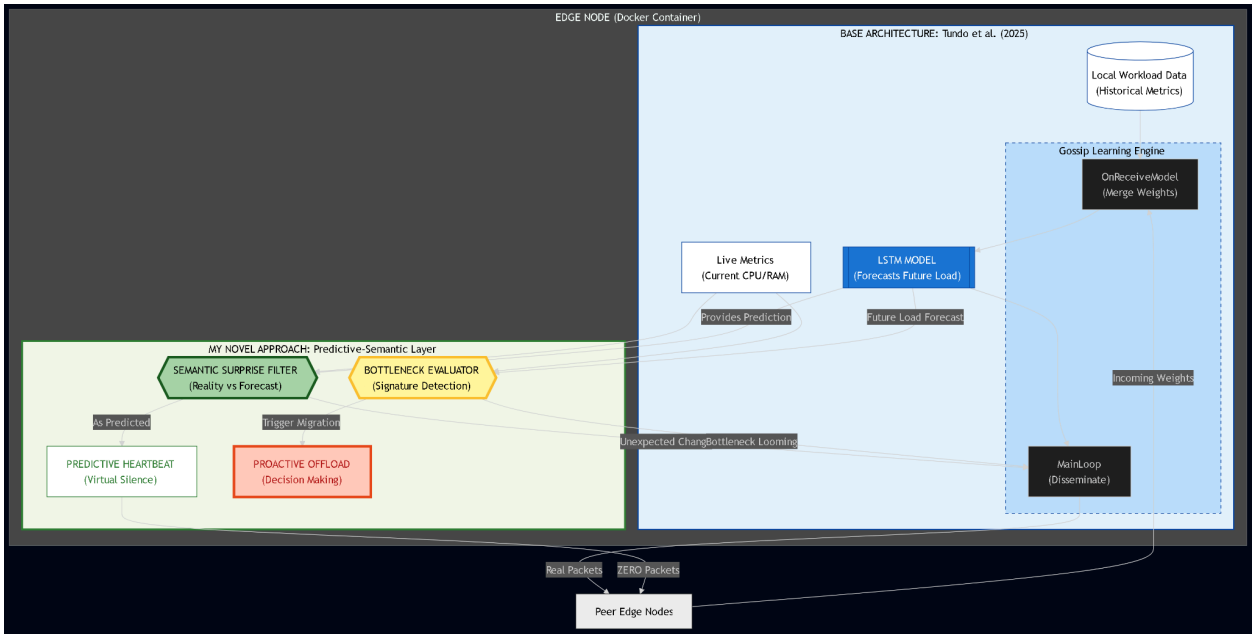
```
Bs = L(i,t) + ( Σ(k=1→H) γk × Δw_hat(i,t+k) ) > α × Ci
```

Where:

- Bs = bottleneck signature
- L(i,t) = current node load
- Δw_hat(i,t+k) = predicted workload increase
- γk = temporal weighting factor
- α = safety coefficient (typically 0.85)
- Ci = node capacity
- H = forecast horizon

Success Benchmarks & Metrics

Metric	Definition	Target Goal
Semantic Compression Gain (SCG)	Ratio of suppressed packets	> 60% savings
Prediction Accuracy (MSE)	Compared to baseline	Within ±5%
Bottleneck Lead Time	Detection before overload	≥ 15 sec
PSP Drift Error	Prediction vs actual MAE	≤ τ



1. The Blue Box (Base Paper - Tundo et al. 2025):
    - Purpose: This layer focuses on Learning.
    - Logic: It uses Gossip Learning to train an LSTM model across all nodes. This allows every node to have a "World Model" that can forecast future traffic patterns.
2. The Green Box (Novel M.Tech Layer):
    - The Semantic Filter: This is the "Traffic Saver." It compares live metrics to the LSTM forecast. If the model is correct, the node stays silent (Predictive Heartbeat). This resolves the bandwidth overhead issue.
    - The Bottleneck Evaluator: This is the "Decision Maker." It identifies a "Bottleneck Signature" by looking at current data + the future forecast.

Agentic AI approach :

1. Base Paper 1: The "Information" Layer - Tundo et al. (2025) - Decentralized Edge Workload Forecasting With Gossip Learning (IEEE TNSM)

- This paper teaches us how to give every edge node a Predictive Eye.
- What we take from it: We use their "Gossip Learning" method. Nodes talk to their neighbors to build a shared LSTM model. This model tells the node: "I predict a 90% traffic spike in 10 seconds."
- The Gap: This paper only gives the information. It doesn't tell the node what to do with it.

2. Base Paper 2: The "Action" Layer - Multi-Agent Reinforcement Learning for Adaptive Resource Orchestration in Cloud-Native Clusters

- This paper teaches us how to give every node a Decision-Making Brain using Reinforcement Learning (RL).
- What we take from it: We use their "Agentic" structure. Each node is an agent that earns "rewards" for making good decisions (like saving battery or finishing tasks fast).
- The Gap: This paper assumes the nodes are in a high-speed Cloud cluster. It doesn't account for the fact that on the Edge, talking (Gossip) is expensive and uses battery.

Novel Approach: "Predictive-Semantic Agentic Orchestration"

In existing work, RL agents make decisions based on what is happening NOW. Your agent makes decisions based on what will happen LATER (using the LSTM forecast).
How it works in 3 Simple Steps:

1. Predict (The Eye): The LSTM (from Tundo) predicts: "A bottleneck is coming."
2. Evaluate (The Manners): The node checks the Semantic Value. If the prediction matches reality, it stays Silent to save bandwidth (Your novelty).
3. Act (The Brain): If a bottleneck is predicted, the RL Agent (inspired by Yao) decides the best action:
    - Action A: Move tasks to Neighbor B.
    - Action B: Lower the CPU power to save energy.
    - Action C: Broadcast an emergency update.

Why this satisfies the Professor's "Agentic AI" requirement:

You should mention these 3 Agentic Features in your doc:

1. Autonomy: Each node manages itself. There is no "Master" server.
2. Proactivity: The node doesn't wait to crash; it acts 20 seconds early because of the LSTM.
3. Learning: The RL agent gets better over time. It learns that "Whenever I offload to Neighbor C, the network gets slow, so I will start offloading to Neighbor D instead."

Summary:

We integrate the Decentralized Gossip Learning from Tundo et al. (2025) to provide high-accuracy workload forecasting. We then feed these forecasts into an Agentic Decision Engine based on the Multi-Agent Reinforcement Learning (MARL) framework proposed by Yao et al. (2025).

Our framework use Forecasted Bottleneck Signatures as the primary state input for a Reinforcement Learning agent at the Edge. This allows for "Predictive Silence" (Semantic Filtering) to save bandwidth while enabling proactive task migration before SLA violations occur.

While this approach introduces local AI agents, it is designed for a net-positive energy gain. On edge hardware, radio communication (bandwidth) is the primary source of battery depletion, often consuming 100s times more energy than CPU computation. Our framework uses local intelligence to 'buy silence,' trading a few CPU cycles for a massive reduction in network transmissions, thereby significantly extending device longevity.